ტექნიკური ანგარიში: შინაური ცხოველების ჯიშის ამოცნობა (Computer Vision)

ავტორი:მათე როგავა კოდი: https://github.com/matesgit/pet-vision ·

1. რა პრობლემას წყვეტს პროექტი?

პროექტი სურათზე ცნობს კატის ან ძაღლის ჯიშს 37 შესაძლო ჯიშიდან (მრავალკლასიანი სურათების კლასიფიკაცია). პრაქტიკული გამოყენება: ვეტერინარული და თავშესაფრის აპები, ცხოველების კატალოგები, ფოტოების ავტომატური მარკირება. ამოცანის განსაკუთრებული სირთულე ისაა, რომ ბევრი ჯიში ერთმანეთს ძალიან ჰგავს (fine-grained classification), ამიტომ მარტივი წესებით ამოცნობა შეუძლებელია და მანქანური სწავლება ბუნებრივი არჩევანია.

2. მონაცემები და რატომ ავირჩიე ისინი

გამოვიყენე Oxford-IIIT Pet Dataset: 7 349 ფოტო, 37 ჯიში (12 კატის და 25 ძაღლის), დაახლოებით 200 ფოტო თითო ჯიშზე. ნაკრები ღიაა და დავალებაში მითითებულ ღია ნაკრებებს შორისაა. არჩევის მიზეზები: საკმარისად მცირეა უფასო Colab GPU-ზე სწრაფი ექსპერიმენტებისთვის, კლასები დაახლოებით ბალანსირებულია და ამოცანა საკმარისად რთულია, რომ მიდგომების შედარებას აზრი ჰქონდეს.

დაყოფა: ოფიციალური trainval ნაწილი (3 680 ფოტო) გავყავი 85% train (3 128) და 15% validation (552) ნაწილად ფიქსირებული seed-ით. ოფიციალური test ნაწილი (3 669 ფოტო) გამოვიყენე მხოლოდ საბოლოო შეფასებისთვის, საუკეთესო epoch კი validation-ით ავირჩიე, რომ test-ზე არჩევანი არ გამეკეთებინა (data leakage).

3. მიდგომა და მოდელები

მთავარი იდეა transfer learning-ია: ~3 000 სასწავლო ფოტო ცოტაა ნულიდან სასწავლად, ამიტომ ავიღე ResNet18, რომელიც ImageNet-ზე უკვე გაწვრთნილი იყო. ბოლო ფენა შევცვალე 37 კლასზე. სამი მოდელი შევადარე:

- **SimpleCNN (ნულიდან)**: baseline, 1.2 მლნ პარამეტრი, რომ გამოჩნდეს pretrained წონების სარგებელი.

- **ResNet18 (pretrained)**: ძირითადი მოდელი, 11.2 მლნ პარამეტრი. პატარაა, სტაბილურია და კარგად შესწავლილი.
- **MobileNetV3-Small (pretrained)**: მსუბუქი მოდელი, 1.6 მლნ პარამეტრი, სიზუსტისა და სიჩქარის კომპრომისის შესამოწმებლად.

ტრენინგის პარამეტრები: Loss არის CrossEntropy, optimizer AdamW, learning rate მცირდება cosine გრაფიკით, გამოთვლები mixed precision-ით (AMP), სურათის ზომა 224×224, batch 32.

4. ექსპერიმენტები

გავუშვი 6 კონტროლირებადი ექსპერიმენტი. ყველა ერთი seed-ით (42) და ერთსა და იმავე დაყოფაზე.

ექსპერიმენტი	კონფიგურაცია	Val acc	Test acc	Macro-F1
scratch_cnn	SimpleCNN, pretrained-ის გარეშე, 30 epoch	44.7%	35.3%	0.342
resnet18_linear	ResNet18, მხოლოდ ბოლო ფენა ისწავლება, 10 epoch	90.9%	88.9%	0.888
resnet18_ft_noaug	სრული fine-tuning, augmentation-ის გარეშე	90.6%	88.4%	0.880
resnet18_ft_basic	სრული fine-tuning, ძირითადი augmentation	92.6%	88.4%	0.883
resnet18_ft_strong	სრული fine-tuning, ძლიერი augmentation	91.7%	88.9%	0.887
mobilenet_ft_basic	MobileNetV3-Small, ძირითადი augmentation	87.1%	85.0%	0.848

თანმხლები ერთეპოქიანი სატესტო გაშვება (ResNet18, 1 epoch) test-ზე 77.5%-ს იძლეოდა.

დასკვნები:

1. **Transfer learning გადამწყვეტია.** Pretrained ResNet18-მა test-ზე 88–89% მიიღო, ნულიდან გაწვრთნილმა მოდელმა მხოლოდ 35%. ~3 000 ფოტო ნულიდან სასწავლად არასაკმარისია, pretrained მოდელი კი ზოგადი ვიზუალური ნიშნების

(კიდეები, ტექსტურები, ფორმები) ცოდნით იწყებს. Scratch მოდელის train accuracy 30 epoch-ის ბოლოსაც მხოლოდ 48% იყო და ისევ იზრდებოდა, ანუ უფრო underfitting იყო, ვიდრე overfitting.

2. **Linear probe თითქმის იგივეს იძლევა, რასაც სრული fine-tuning.** ბოლო ფენის დატრენინგებამ (მხოლოდ 18 981 პარამეტრი, მოდელის 0.17%) 88.9% მოიტანა, 11.2 მლნ პარამეტრის დატრენინგებამ კი 88.4–88.9%. სავარაუდო მიზეზი: ImageNet-ში უკვე ბევრი კატა და ძაღლია, ამიტომ pretrained ნიშნები ამ ამოცანისთვის უკვე კარგია და დანარჩენი ქსელის მორგება დამატებით მცირე სარგებელს იძლევა.
3. **Augmentation-მა test accuracy ვერ გააუმჯობესა.** სამი ვარიანტი 88.4%, 88.4% და 88.9%-ს იძლევა და ეს სხვაობა ცდომილების ფარგლებშია (test-ში 3 669 ფოტოა, ერთი seed). ეფექტი overfitting-ზე კი ჩანს: augmentation-ის გარეშე train accuracy 5-ე epoch-ზე 100%-ს აღწევს (მოდელი ფოტოებს ზეპირობს), ძირითადი augmentation-ით validation loss 0.299-დან 0.261-მდე დაეცა. ძლიერი augmentation train-ს ანელებს (train acc 97.6%), მაგრამ epoch 1.7-ჯერ უფრო ნელია (30 წმ vs 18 წმ), რადგან სურათების დამუშავება მეტ დროს მოითხოვს. პატიოსანი ფორმულირება: ამ ექსპერიმენტით ვერ დავადასტურე, რომ augmentation accuracy-ს ზრდის.
4. **MobileNet მცირე სიზუსტის ფასად ბევრად მსუბუქია:** 6.0 MB vs 42.7 MB და დაახლოებით 3.4 პუნქტით დაბალი test accuracy.

საბოლოო მოდელად ავირჩიე **resnet18_ft_basic**, რადგან validation accuracy-ით ის საუკეთესოა (92.6%). test-ის შედეგით არჩევა მეთოდოლოგიურად არასწორი იქნებოდა. test შედეგები ერთმანეთთან ძალიან ახლოსაა, ამიტომ არჩევანი მყარ უპირატესობას კი არა, validation-ზე დაფუძნებულ გადაწყვეტას ეფუძნება.

5. შეფასება და შეცდომების ანალიზი

საბოლოო მოდელი (resnet18_ft_basic), test ნაკრებზე: accuracy 88.4%, top-5 accuracy 98.8%, macro-F1 0.883. Macro-F1 ყველა კლასს თანაბრად აფასებს, top-5 კი გვაჩვენებს, რომ სწორი ჯიში თითქმის ყოველთვის პირველ ხუთ ვარიანტშია.

ყველაზე ხშირი შეცდომები (ყველა გაშვებაში ერთი და იგივე წყვილები მეორდება):

- American Pit Bull Terrier გამოცხადდა Staffordshire Bull Terrier-ად ან American Bulldog-ად
- Staffordshire Bull Terrier გამოცხადდა American Bulldog-ად არაერთხელ
- Ragdoll და Birman ერთმანეთში ერევა

ყველაზე რთული კლასები (recall): American Pit Bull Terrier 31–47%, Staffordshire Bull Terrier 50–65%, ასევე Ragdoll, Birman და Maine Coon (70–80%).

ამ შეცდომებს ერთი მახასიათებელი აქვს: ეს ჯიშები ვიზუალურად ძალიან გავს ერთმანეთს, ზოგჯერ ადამიანისთვისაც ძნელი გასარჩევია (ორი გრძელბეწვიანი ცისფერთვალება კატა, სამი მსგავსი ბულტერიერი). ანუ მოდელი სუსტდება იქ, სადაც ამოცანა თავად არის ბუნდოვანი. ის ფაქტი, რომ ყველა მოდელს (scratch-ის გარდა) იგივე წყვილები ერევა, მიანიშნებს, რომ პრობლემა დატრენინგების მეთოდზე კი არა, კლასების მსგავსებასა და მონაცემთა სიმცირეზეა (~200 ფოტო თითო ჯიშზე).

6. ოპტიმიზაცია

სამი მიმართულებით შევამოწმე ეფექტურობა: mixed precision (AMP) ტრენინგში, უფრო მსუბუქი არქიტექტურა (MobileNet) და ONNX Runtime ინფერენსისთვის. ონლაინ გაზომვა გავაკეთე CPU-ზე, ერთი სურათი (batch=1):

მოდელი	ზომა	PyTorch CPU	ONNX Runtime CPU	აჩქარება	მაქს. სხვაობა გამოსავალში
ResNet18	42.7 MB	59.4 ms	33.8 ms	1.8×	1.1e-6
MobileNetV3-Small	5.9 MB	9.4 ms	2.3 ms	4.0×	3.0e-6

ONNX Runtime-მა პროგნოზი პრაქტიკულად უცვლელად დატოვა (სხვაობა მილიონიან ნაწილზე ნაკლებია) და ინფერენსი 1.8–4-ჯერ დააჩქარა. MobileNet + ONNX Runtime ResNet18 + PyTorch-ს დაახლოებით 25-ჯერ ჯობია სიჩქარით და 7-ჯერ ზომით, ხოლო სიზუსტე 3.4 პუნქტით ნაკლებია. დასკვნა: სერვერზე, სადაც სიზუსტე უფრო მნიშვნელოვანია, ResNet18 უფრო ზუსტი არჩევანია, მობილურ ან სუსტ მოწყობილობაზე კი MobileNet + ONNX უკეთესი კომპრომისია. შენიშვნა: evaluate სკრიპტის GPU-ზე გაზომილი latency (3–5 ms) ხმაურიანია და ამ ანგარიშში არ ვიყენებ.

7. დემო აპლიკაცია

ავაწყვე დემოს ორი ვერსია:

1. ცოცხალი ვებ-დემო (მუშაობს ბრაუზერში, ინსტალაციის გარეშე): https://huggingface.co/spaces/mateshuggingface/pet-breed-classifier

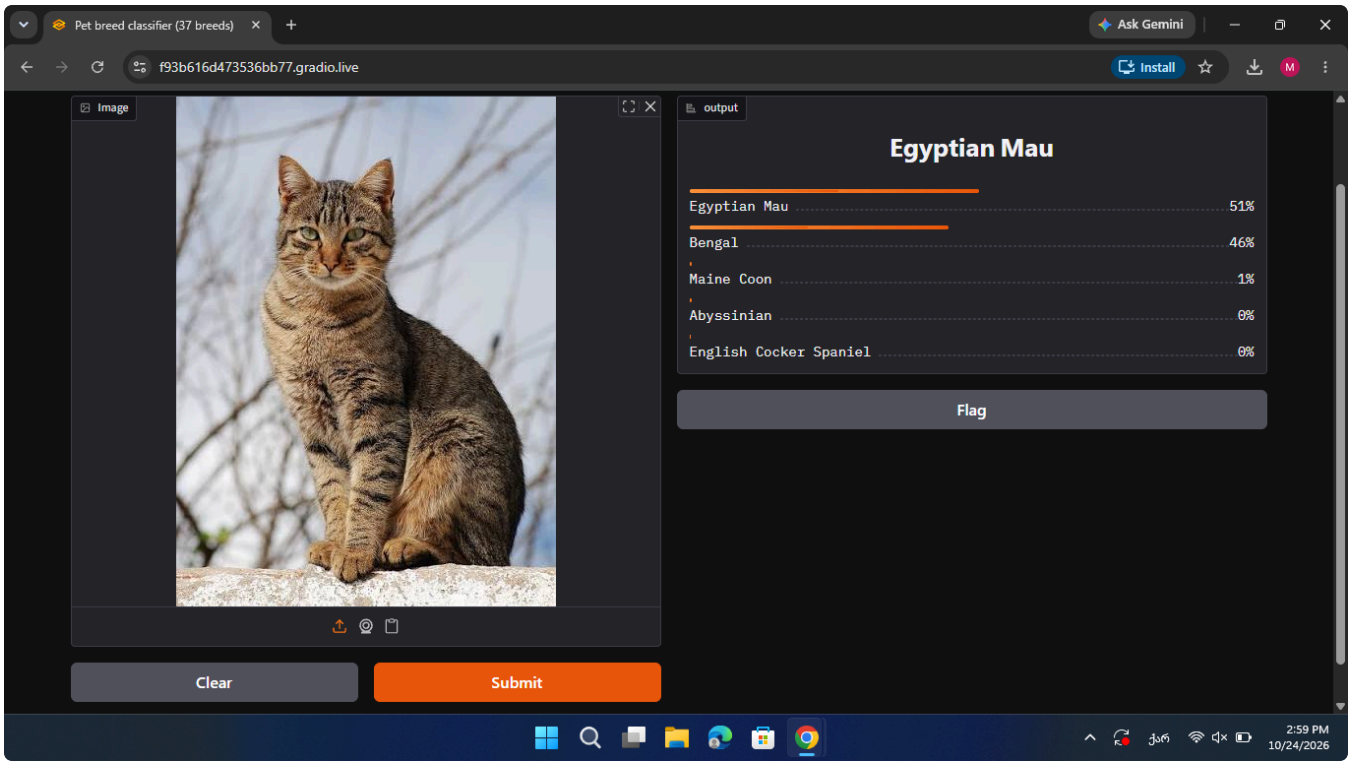
ის მუშაობს უშუალოდ ბრაუზერში (ONNX Runtime Web) და იყენებს იმავე ResNet18 მოდელს, რომელიც ანგარიშში საბოლოო მოდელადაა დასახელებული. ფოტო არსად იტვირთება, ყველა გამოთვლა მომხმარებლის მოწყობილობაზე სრულდება. ბრაუზერში

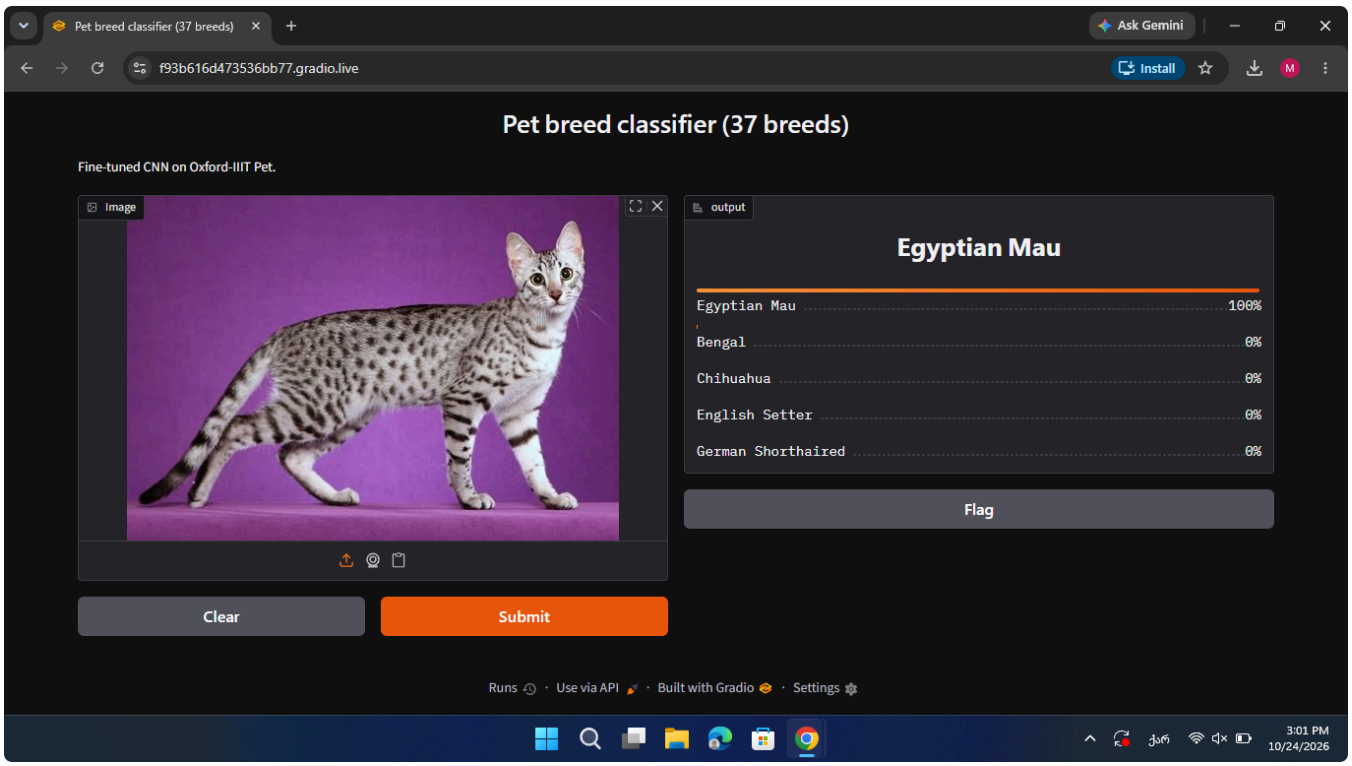
სურათის შემცირების ალგორითმი Python-ისგან ოდნავ განსხვავდება, ამიტომ პროცენტები რამდენიმე წილით შეიძლება არ დაემთხვეს (მაგ. Chihuahua: 90.1% ბრაუზერში და 92.2% Python-ში), თუმცა პროგნოზირებული ჯიში იგივეა.

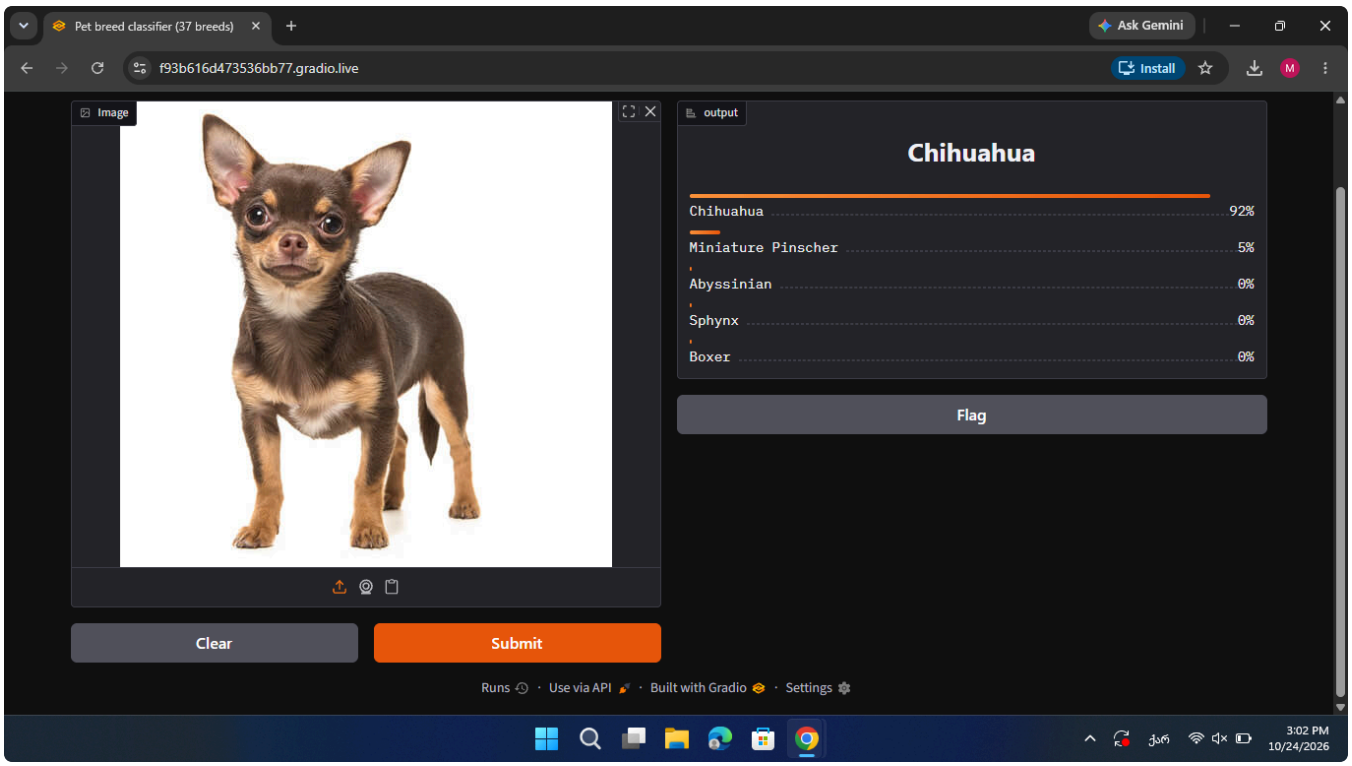
2.Gradio-ს ვერსია (Colab-ში გასაშვები, `app.py`): მომხმარებელი ტვირთავს ფოტოს და ხედავს top-5 ჯიშს პროცენტებით. სამ ფოტოზე გამოვცადე:

- Egyptian Mau-ს სტუდიური ფოტო: სწორად ამოიცნო, Egyptian Mau 100%.
- Chihuahua: სწორად ამოიცნო, Chihuahua 92% (მეორე ვარიანტი Miniature Pinscher 5%, ანუ ლოგიკური).
- ჩვეულებრივი ზოლებიანი (tabby) კატა: Egyptian Mau 51% / Bengal 46%. ამ ჯიშის კლასი 37 ჯიშში არ არის, ამიტომ მოდელი ორ უახლოეს ჯიშს შორის ყოყმანობს.

სკრინშოტები მაგალითისთვის:




Pet breed classifier (37 breeds)
Ask Gemini
f93b616d473536bb77.gradio.live
Install
Pet breed classifier (37 breeds)
Fine-tuned CNN on Oxford-IIIT Pet.
Image
output
Egyptian Mau
Egyptian Mau 100%
Bengal 0%
Chihuahua 0%
English Setter 0%
German Shorthaired 0%
Flag
Clear
Submit
Runs · Use via API · Built with Gradio · Settings
3:01 PM
10/24/2026


Pet breed classifier (37 breeds)
Ask Gemini
f93b616d473536bb77.gradio.live
Install
Image
output
Chihuahua
Chihuahua 92%
Miniature Pinscher 5%
Abyssinian 0%
Sphynx 0%
Boxer 0%
Flag
Clear
Submit
Runs · Use via API · Built with Gradio · Settings
3:02 PM
10/24/2026

8. რა პრობლემები შემხვდა და როგორ გადავწყვიტე

- **Colab-ში ფაილების ატვირთვა გაიჭედა.** საქაღალდის ხელით ატვირთვამ არ იმუშავა. გადავწყვიტე ერთი zip ფაილის ატვირთვით და ფაილის ზუსტი მისამართის (`/pet-vision-project.zip`) გამოყენებით, თუმცა აქ დიდად რთული არაფერი იყო
- **შედეგების ინტერპრეტაცია.** ერთმანეთთან ძალიან ახლო test შედეგების გამო (88.4–88.9%) არ შეიძლებოდა დასკვნა, რომ რომელიმე ვარიანტი უკეთესია. გადავწყვიტე, საბოლოო მოდელი validation-ით ამეღო და ეს მკაფიოდ აღმენიშნა.
- რათქმაუნდა ყველა ბრძანებას თუ კოდის ლაინს ზეპირად ვერ დაიმახსოვრებ, ამიტომ მომიწია რამდენიმე ბრძანების გუგლში მოძიება.

9. ჩემი გადაწყვეტის ძლიერი და სუსტი მხარეები

ძლიერი: სუფთა train/validation/test გამიჯვნა, რამდენიმე კონტროლირებადი ექსპერიმენტი ერთი ცვლადის ცვლილებით, შეცდომების ანალიზი, რეპროდუცირებადი კოდი (ერთი ბრძანებით ყველა ექსპერიმენტი) და მუშა დემო.

სუსტი: თითო ექსპერიმენტი მხოლოდ ერთი seed-ით, ამიტომ 0.5 პუნქტის სხვაობას ვერ განვასხვავებ ხმაურისგან. Hyperparameter search არ ჩამიტარებია. Validation ნაწილი stratified არ არის. მოდელი მხოლოდ 37 ჯიშს ცნობს და უცნობ ჯიშზე ზედმეტად თავდაჯერებული შეიძლება იყოს (მაგ. 100%-იანი პროგნოზი ყოველთვის სანდო არ არის). "სხვა/უცნობი" კლასი და უარყოფის მექანიზმი არ არის.

10. დამატებითი დროისა და რესურსების შემთხვევაში რას გავაუმჯობესებდი

1. თითო ექსპერიმენტი 3–5 სხვადასხვა seed-ით და შედეგი საშუალო +- გადახრით, რომ სტატისტიკურად დავასაბუთო, რომელი ვარიანტია უკეთესი.
2. Hyperparameter search (learning rate, weight decay, epoch-ების რაოდენობა), ასევე უფრო ძლიერი არქიტექტურები (EfficientNet, ConvNeXt).
3. რთული წყვილების (Pit Bull / Staffordshire, Ragdoll / Birman) უკეთესი გარჩევისთვის მეტი მონაცემი ან უფრო მაღალი გარჩევადობა.
4. Confidence calibration (მაგ. label smoothing) და „უცნობი ჯიშის" კლასი/ზღვარი დაბალი ნდობის პროგნოზების უარყოფისთვის.
5. Grad-CAM, რომ ვნახო, მოდელი ფოტოზე რას უყურებს ჯიშის ამოცნობისას.
6. INT8 quantization ინფერენსის დამატებითი დასაჩქარებლად, ასევე FastAPI + Docker გაშვებისთვის.

11. AI ინსტრუმენტების გამოყენება

ამ პროექტზე ვიმუშავე Claude-ის დახმარებით. კერძოდ:

- პროექტის დაგეგმარება და საწყისი Python კოდი
- **რამდენიმე ბრძანება და ერორების განხილვა.**
- **შეცდომების გამოსწორება:** გაშვებისას გამოჩენილი შეცდომა (shell-ის არგუმენტების პრობლემა) Claude-თან ერთად გავარკვიე და გავასწორე.
- **ჩემი წვლილი:** ყველა ექსპერიმენტი თავად გავუშვი Colab-ში (T4 GPU), შედეგები ჩემი გაშვებებიდანაა, დემო თავად გამოვცადე ჩემს მიერ არჩეულ ფოტოებზე, ხოლო შედეგების ინტერპრეტაცია და დასკვნები შესაბამისად გავაკეთე.